

PATRÓN DE DISEÑO: ADAPTADOR (ADAPTER)

Capítulo 20 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

En general, los clientes de una clase acceden a los servicios ofrecidos por dicha clase a través de su interfaz. En ocasiones, una clase existente puede proveer la funcionalidad requerida por un cliente, pero su interfaz puede no ser la que el cliente espera. Esto puede ocurrir por diversas razones: la interfaz existente puede ser demasiado detallada, puede carecer de detalle suficiente, o la terminología utilizada puede ser diferente a la que el cliente busca.

En tales casos, la interfaz existente debe convertirse en la interfaz que el cliente espera, preservando la reutilizabilidad de la clase existente. Sin dicha conversión, el cliente no podrá usar la funcionalidad ofrecida por la clase.

Esto puede lograrse mediante el Patrón Adaptador. El patrón sugiere definir una clase envoltorio alrededor del objeto con la interfaz incompatible. Este objeto envoltorio se denomina adaptador (adapter) y el objeto que envuelve se denomina adaptado (adaptee). El adaptador provee la interfaz requerida por el cliente. La implementación de la interfaz del adaptador convierte las llamadas del cliente en llamadas a la interfaz de la clase adaptado. En otras palabras, cuando un cliente invoca un método del adaptador, internamente el adaptador llama a un método del adaptado, del cual el cliente no tiene conocimiento. Esto le otorga al cliente acceso indirecto a la clase adaptado.

Así, un adaptador puede usarse para hacer que clases con interfaces incompatibles trabajen juntas.

El término "interfaz" utilizado en la discusión anterior:

- No se refiere al concepto de interfaz en la programación Java, aunque la interfaz de una clase puede declararse usando una interfaz Java.
- No se refiere a la interfaz de usuario de una aplicación GUI típica con ventanas y controles gráficos.
- Sí se refiere a la interfaz de programación que una clase expone y que está destinada a ser usada por otras clases. Por ejemplo, cuando una clase se declara como clase abstracta o como interfaz Java, el conjunto de métodos definidos en ella constituye la interfaz de la clase.

2. Adaptadores de Clase versus Adaptadores de Objeto

Los adaptadores pueden clasificarse en dos categorías — adaptadores de clase y adaptadores de objeto — según la forma en que se diseña el adaptador.

2.1 Adaptador de Clase

Un adaptador de clase se diseña subclasificando la clase adaptado. Además, el adaptador implementa la interfaz que el objeto cliente espera. Cuando el cliente invoca un método del adaptador de clase, el adaptador llama internamente al método heredado del adaptado.

2.2 Adaptador de Objeto

Un adaptador de objeto contiene una referencia a un objeto adaptado. Al igual que el adaptador de clase, el adaptador de objeto también implementa la interfaz que el cliente espera. Cuando el cliente invoca un método del adaptador de objeto, este llama al método apropiado sobre la instancia del adaptado cuya referencia contiene.

La Tabla 1 lista en detalle las diferencias entre adaptadores de clase y adaptadores de objeto.

Tabla 1 — Adaptadores de Clase versus Adaptadores de Objeto

Adaptadores de Clase	Adaptadores de Objeto
Basado en el concepto de herencia.	Utiliza composición de objetos.
Puede usarse para adaptar la interfaz del adaptado únicamente. No puede adaptar las interfaces de sus subclases, ya que el adaptador está enlazado estáticamente con el adaptado al momento de su creación.	Puede usarse para adaptar la interfaz del adaptado y todas sus subclases.
Dado que el adaptador es diseñado como subclase del adaptado, es posible sobrescribir parte del comportamiento del adaptado. (Nota: En Java, una subclase no puede sobrescribir un método declarado como final en su clase padre.)	No puede sobrescribir métodos del adaptado. (Nota: Literalmente no puede "sobrescribir" ya que no hay herencia; sin embargo, las funciones envoltorio del adaptador pueden cambiar el comportamiento según sea necesario.)
El cliente tendrá cierto conocimiento de la interfaz del adaptado, ya que la interfaz pública completa del adaptado es visible para el cliente.	El cliente y el adaptado están desacoplados. Solo el adaptador conoce la interfaz del adaptado.
En aplicaciones Java: Adecuado cuando la interfaz esperada está disponible como interfaz Java y no como clase abstracta o concreta, ya que Java solo permite herencia simple. Dado que el adaptador de clase es	En aplicaciones Java: Adecuado incluso cuando la interfaz que el cliente espera está disponible como clase abstracta. También puede usarse si la interfaz esperada está en

Adaptadores de Clase	Adaptadores de Objeto
subclase del adaptado, no podrá también subclasificar la clase de interfaz si esta está disponible como clase abstracta o concreta.	forma de interfaz Java, o cuando se necesita adaptar el adaptado y todas sus subclases.
En aplicaciones Java: Puede adaptar métodos con especificador de acceso protected.	En aplicaciones Java: No puede adaptar métodos con especificador de acceso protected, a menos que el adaptador y el adaptado estén diseñados en el mismo paquete.

3. Ejemplo — Validación de Dirección de Clientes

Construyamos una aplicación para validar una dirección de cliente determinada. Esta aplicación puede ser parte de una aplicación más grande de gestión de datos de clientes.

Se define una clase Customer (Cliente). Diferentes objetos cliente pueden crear un objeto Customer e invocar el método isValidAddress para verificar la validez de la dirección del cliente. Para el proceso de validación, la clase Customer espera hacer uso de una clase validadora que provea la interfaz declarada en la interfaz AddressValidator.

Se define un validador USAddress para validar una dirección estadounidense. La clase USAddress está diseñada para implementar la interfaz AddressValidator, de modo que los objetos Customer puedan usar instancias de USAddress en el proceso de validación sin ningún problema.

Listado 3.1 — Clase Customer

```
class Customer {
    public static final String US = "US";
    public static final String CANADA = "Canada";
    private String address;
    private String name;
    private String zip, state, type;

    public boolean isValidAddress() {
        // ...
    }

    public Customer(String inp_name, String inp_address,
                    String inp_zip, String inp_state,
                    String inp_type) {
        name = inp_name;
        address = inp_address;
        zip = inp_zip;
        state = inp_state;
        type = inp_type;
    }
}
```

```
} // fin de la clase
```

Listado 3.2 — AddressValidator como Interfaz

```
public interface AddressValidator {  
    public boolean isValidAddress(String inp_address,  
                                   String inp_zip,  
                                   String inp_state);  
} // fin de la clase
```

Listado 3.3 — Clase USAddress

```
class USAddress implements AddressValidator {  
    public boolean isValidAddress(String inp_address,  
                                   String inp_zip,  
                                   String inp_state) {  
        if (inp_address.trim().length() < 10) return false;  
        if (inp_zip.trim().length() < 5) return false;  
        if (inp_zip.trim().length() > 10) return false;  
        if (inp_state.trim().length() != 2) return false;  
        return true;  
    }  
} // fin de la clase
```

Listado 3.4 — Clase Customer usando USAddress

```
class Customer {  
    // ...  
    public boolean isValidAddress() {  
        AddressValidator validator = getValidator(type);  
        return validator.isValidAddress(address, zip, state);  
    }  
    private AddressValidator getValidator(String custType) {  
        AddressValidator validator = null;  
        if (custType.equals(Customer.US)) {  
            validator = new USAddress();  
        }  
        return validator;  
    }  
} // fin de la clase
```

3.1 El Problema: Interfaz Incompatible

Supongamos que la aplicación necesita extenderse para tratar también con clientes de Canadá. Esto requiere un validador para verificar las direcciones de clientes canadienses. Asumamos que ya existe una clase utilitaria CAAddress con la funcionalidad necesaria para validar una dirección canadiense.

De la implementación de la clase CAAddress puede observarse que esta sí ofrece el servicio de validación requerido por la clase Customer, pero la interfaz que ofrece es diferente a la que Customer espera. La clase CAAddress ofrece un método isValidCanadianAddr, mientras que

Customer espera un método isValidAddress tal como se declara en la interfaz AddressValidator.

Esta incompatibilidad en la interfaz dificulta que un Customer pueda usar la clase CAAddress existente. Una opción sería modificar la interfaz de la clase CAAddress, pero no es recomendable porque podría haber otros clientes que usan CAAddress en su forma actual, y cambiarla afectaría a todos ellos.

Listado 3.5 — Clase CAAddress con Interfaz Incompatible

```
class CAAddress {
    public boolean isValidCanadianAddr(String inp_address,
                                      String inp_pcode,
                                      String inp_prvnc) {
        if (inp_address.trim().length() < 15) return false;
        if (inp_pcode.trim().length() != 6)   return false;
        if (inp_prvnc.trim().length() < 6)    return false;
        return true;
    }
} // fin de la clase
```

4. Adaptador de Dirección como Adaptador de Clase

Aplicando el Patrón Adaptador, se diseña un adaptador de clase CAAddressAdapter como subclase de la clase CAAddress implementando la interfaz AddressValidator.

Dado que el adaptador CAAddressAdapter implementa la interfaz AddressValidator, los objetos cliente pueden acceder al adaptador sin ningún problema. Cuando un objeto cliente invoca el método isValidAddress sobre la instancia del adaptador, el adaptador lo traduce internamente en una llamada al método isValidCanadianAddr.

Dentro de la clase Customer, el método privado getValidator se mejora para retornar una instancia de CAAddressAdapter para los clientes canadienses. La llamada polimórfica sobre el resultado (dentro del método isValidAddress) no necesita modificarse, ya que tanto USAddress como CAAddressAdapter implementan la misma interfaz AddressValidator.

Listado 4.1 — CAAddressAdapter como Adaptador de Clase

```
public class CAAddressAdapter extends CAAddress
    implements AddressValidator {
    public boolean isValidAddress(String inp_address,
                                String inp_zip,
                                String inp_state) {
        return isValidCanadianAddr(inp_address,
                                inp_zip, inp_state);
    }
} // fin de la clase
```

Listado 4.2 — Clase Customer usando CAAddressAdapter

```
class Customer {
    // ...
    public boolean isValidAddress() {
        AddressValidator validator = getValidator(type);
        return validator.isValidAddress(address, zip, state);
    }
    private AddressValidator getValidator(String custType) {
        AddressValidator validator = null;
        if (custType.equals(Customer.US)) {
            validator = new USAddress();
        }
        if (type.equals(Customer.CANADA)) {
            validator = new CAAddressAdapter();
        }
        return validator;
    }
} // fin de la clase
```

La combinación del diseño del CAAddressAdapter y la llamada polimórfica sobre un objeto del tipo AddressValidator permite a la clase Customer hacer uso de los servicios de la clase CAAddress, que tiene una interfaz incompatible.

5. Adaptador de Dirección como Adaptador de Objeto

Durante la discusión del diseño del adaptador de dirección como adaptador de clase, la interfaz AddressValidator esperada por el cliente estaba disponible en forma de interfaz Java. Ahora supongamos que el cliente espera que la interfaz AddressValidator esté disponible como clase abstracta en lugar de interfaz Java.

Dado que el adaptador CAAddressAdapter debe proveer los métodos declarados por la clase abstracta AddressValidator, el adaptador debe diseñarse como subclase de dicha clase abstracta e implementar sus métodos abstractos.

Sin embargo, como la herencia múltiple no está soportada en Java, el adaptador CAAddressAdapter ya no puede subclasificar la clase CAAddress existente, ya que habrá usado su única oportunidad de subclasificar otra clase.

Aplicando el Patrón Adaptador de Objeto, el CAAddressAdapter puede diseñarse para contener una instancia del adaptado CAAddress. Esta instancia del adaptado es pasada al adaptador por sus clientes cuando se crea por primera vez.

En general, la instancia del adaptado contenida por un adaptador de objeto puede proporcionarse de las siguientes dos maneras:

- Los clientes del adaptador de objeto pueden pasar la instancia del adaptado al adaptador. Este enfoque es más flexible en la elección de la clase a adaptar, aunque el

cliente puede llegar a tener conocimiento del adaptado o del hecho de la adaptación. Es más adecuado cuando el adaptador necesita algún estado específico del objeto adaptado además de su comportamiento.

- El adaptador puede crear la instancia del adaptado por sí mismo. Este enfoque es relativamente menos flexible y adecuado cuando el adaptador no necesita un estado específico del adaptado, sino solo su comportamiento.

Listado 5.1 — AddressValidator como Clase Abstracta

```
public abstract class AddressValidator {
    public abstract boolean isValidAddress(String inp_address,
                                           String inp_zip,
                                           String inp_state);
} // fin de la clase
```

Listado 5.2 — CAAddressAdapter como Adaptador de Objeto

```
class CAAddressAdapter extends AddressValidator {
    private CAAddress objCAAddress;

    public CAAddressAdapter(CAAddress address) {
        objCAAddress = address;
    }

    public boolean isValidAddress(String inp_address,
                                  String inp_zip,
                                  String inp_state) {
        return objCAAddress.isValidCanadianAddr(
            inp_address, inp_zip, inp_state);
    }
} // fin de la clase
```

Cuando un objeto cliente invoca el método `isValidAddress` sobre la instancia del adaptador `CAAddressAdapter`, el adaptador llama internamente al método `isValidCanadianAddr` sobre la instancia del adaptado `CAAddress` que contiene.

Del diseño de la aplicación de ejemplo puede observarse que un adaptador permite a la clase `Customer` (cliente) acceder a los servicios ofrecidos por la clase `CAAddress` (adaptado) que tiene una interfaz incompatible.

6. Preguntas de Práctica

1. Durante la discusión del patrón `Factory Method`, se diseñó una clase de registro de mensajes `FileLogger` con un método `log(String)` que puede ser utilizado por objetos cliente para registrar mensajes. Supongamos que un cliente `LoggerClient` espera que la clase de registro de mensajes provea una interfaz de la siguiente manera:

```
public abstract class LoggerIntr {
    public abstract boolean logMessage(String msg);
} // fin de la clase
```

¿Cómo diseñaría un adaptador, denominado FileLoggerAdapter, para adaptar la interfaz existente de la clase FileLogger?

2. En la pregunta de práctica anterior, si el cliente LoggerClient espera que la clase de registro de mensajes provea una interfaz de la siguiente manera:

```
public interface LoggerIntr {
    public boolean logMessage(String msg);
} // fin de la interfaz
```

¿Puede diseñar el adaptador FileLoggerAdapter como adaptador de clase, como adaptador de objeto, o como ambos?

3. Diseñe dos subclases — HTMLFileLogger y EncFileLogger — de la clase FileLogger según la Tabla 2. Cada una de estas subclases sobrescribe el método log(String) de la clase padre FileLogger para agregar la funcionalidad requerida. Asuma que el cliente LoggerClient espera la funcionalidad ofrecida por FileLogger y también por sus subclases, y que espera que la clase de registro de mensajes provea la siguiente interfaz:

```
public interface LoggerIntr {
    public boolean logMessage(String msg);
} // fin de la interfaz
```

¿Puede diseñar el adaptador FileLoggerAdapter como adaptador de clase, como adaptador de objeto, o como ambos? ¿Por qué?

Tabla 2 — Subclases de FileLogger

Subclase	Funcionalidad
HTMLFileLogger	Transforma un mensaje entrante en un documento HTML y lo almacena en un archivo de log.
EncFileLogger	Aplica encriptación a un mensaje entrante y lo almacena en un archivo de log.